

3D Vehicle Trajectory Recovery and Reconstruction from Uncalibrated Moving Drone Footage

1. Problem Statement

The goal is to extract the 3D spatial trajectory taken by a moving vehicle from a single, uncalibrated, moving drone video and visualize that trajectory within a 3D reconstructed version of the scene in video.

Application: Surveillance, Forensics, Offline Scene Processing, Virtual Reality

2. Introduction

Accurate 3D reconstruction and object tracking from UAV video is a core challenge in computer vision and autonomous systems. Urban scenes captured from drones are inherently dynamic — moving vehicles violate the static scene assumption underlying most reconstruction pipelines, introducing ghosting artifacts and corrupting geometric consistency. Traditional Structure from Motion (SfM) [2] techniques recover accurate camera poses and sparse geometry but treat the scene as entirely static, providing no mechanism for separating dynamic objects from the background.

Neural Radiance Fields (NeRF) [3] advances the possibilities in the field of photorealistic scene reconstruction significantly, but their computational cost limits real-time applicability. 3D Gaussian Splatting (3DGS) [5] addressed this by representing scenes as explicit 3D Gaussian primitives rendered via differentiable rasterisation, achieving comparable quality to NeRF at interactive frame rates. Recent work has extended 3DGS to dynamic UAV footage [1], explicitly handling moving objects, while the nerfstudio framework [6] provides accessible, modular implementations of these pipelines for rapid experimentation on custom data.

Extracting vehicle trajectories from the reconstructed scene introduces a further challenge: projecting detections from image space to 3D world coordinates requires resolving depth ambiguity from a single moving camera. Monocular depth estimation models such as Depth Anything V2 [4] provide a data-driven alternative to purely geometric approaches.

This project extracts and visualises the 3D trajectory of a moving vehicle from monocular drone video within a photorealistic 3DGS digital twin. The pipeline combines SfM-based camera localisation [2], 3DGS scene reconstruction [5, 6], YOLOv8 vehicle detection and tracking, geometric ground-plane projection via per-frame analytical homographies, and metric depth estimation [4] as a complementary localisation approach. The trajectory is visualised directly within the trained 3DGS scene for qualitative evaluation against the reconstructed road surface.

2. Paper Implementation

The 2025 paper by Huang et al. [1] presents a complete pipeline for reconstructing urban 3D environments from a single full-motion video while accurately modelling moving objects. Their approach integrates scene reconstruction, vehicle tracking, refined depth estimation and 4D Gaussian splatting. The paper proposes several modules that derive the necessary information

for the dynamic 3DGS framework directly from the video data, thereby eliminating the need for auxiliary input data like prior knowledge of camera calibration and poses, the 3D point clouds of dynamic objects (typically provided by LiDAR), and the 3D bounding boxes of these objects [1]. The availability of auxiliary data can facilitate network training and yield realistic novel view synthesis, however, such prior information is often unavailable in real-life scenarios.

Fig 1 below shows the overview of the pipeline proposed in [1]:

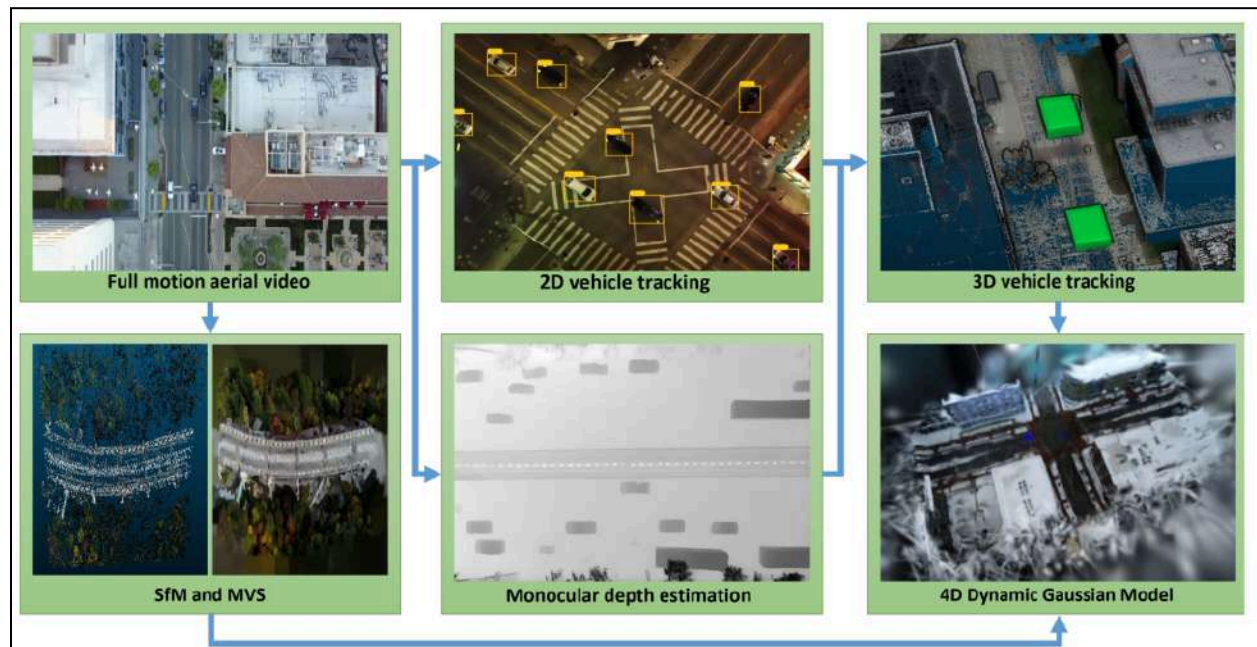


Fig 1: Overview of Pipeline proposed in [1]

This implementation used an input video of 14 seconds where the native FPS of the video is 25 frames per second. The frames were extracted at a fixed rate of 5 FPS, yielding 69 frames as input for the subsequent pipeline. **Fig 2** below shows the composite image of nine equally-spaced frames out of those 69 frames.



Fig 2: 9 equally-spaced frames shown here out of the 69 frames extracted from 14s video (using 5 frames-per-second), showing the overall surrounding scenery in video

The selected frames served as inputs for structure-from-motion (SfM) which estimates camera poses and generates a sparse point cloud that captures the essential geometry of the scene. It employs multi-view stereo (MVS) to create a detailed point cloud that reflects the intricate structure of the urban landscape. This dense point cloud forms the foundational model for the static components of the scene and is subsequently used to initialize the Gaussian field representation. **Fig 3** below shows the screen-shot of the detailed point cloud as viewed in Open3D viewer from four different views.

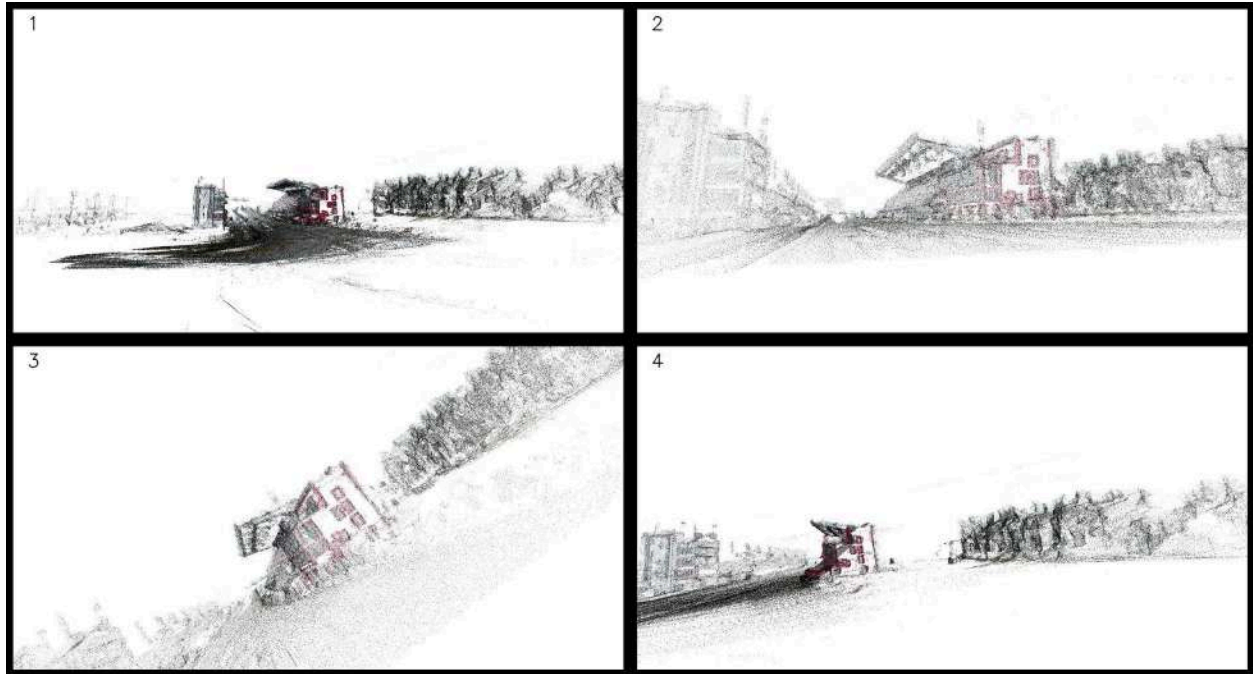


Fig 3: Screen-shot of detailed point cloud viewed in Open3D viewer from 4 different views

To reconstruct the dynamic vehicles, the paper detects the two-dimensional trajectory of each vehicle in the frames and then back-projects these 2D trajectories into three-dimensional space to obtain their precise spatial locations. It uses YOLOv8 object detection network to detect vehicles in each video frame. It then uses BoTSORT tracker [7] to link these detections into continuous 2D trajectories, filling in any missing data through linear interpolation. **Fig 4** below shows the composite image of 4 equally-spaced frames out of 69 frames showing the tracked vehicles.



Fig 4: Composite image of 4 equally-spaced frames, showing the tracked vehicles

As evident in the figure, the BoT-SORT tracker assigns a unique consistent identifier to the vehicle in each frame. This helps in creating a unique 2d track for each vehicle in the video. To extend the trajectories into three dimensions, we use a monocular depth estimation method as proposed in the paper [1]. 'Depth-Anything-V2-Small' model is used to generate the depth maps. **Fig 5** below shows the composite image of 4 equally-spaced depth-maps out of those 69 frames.

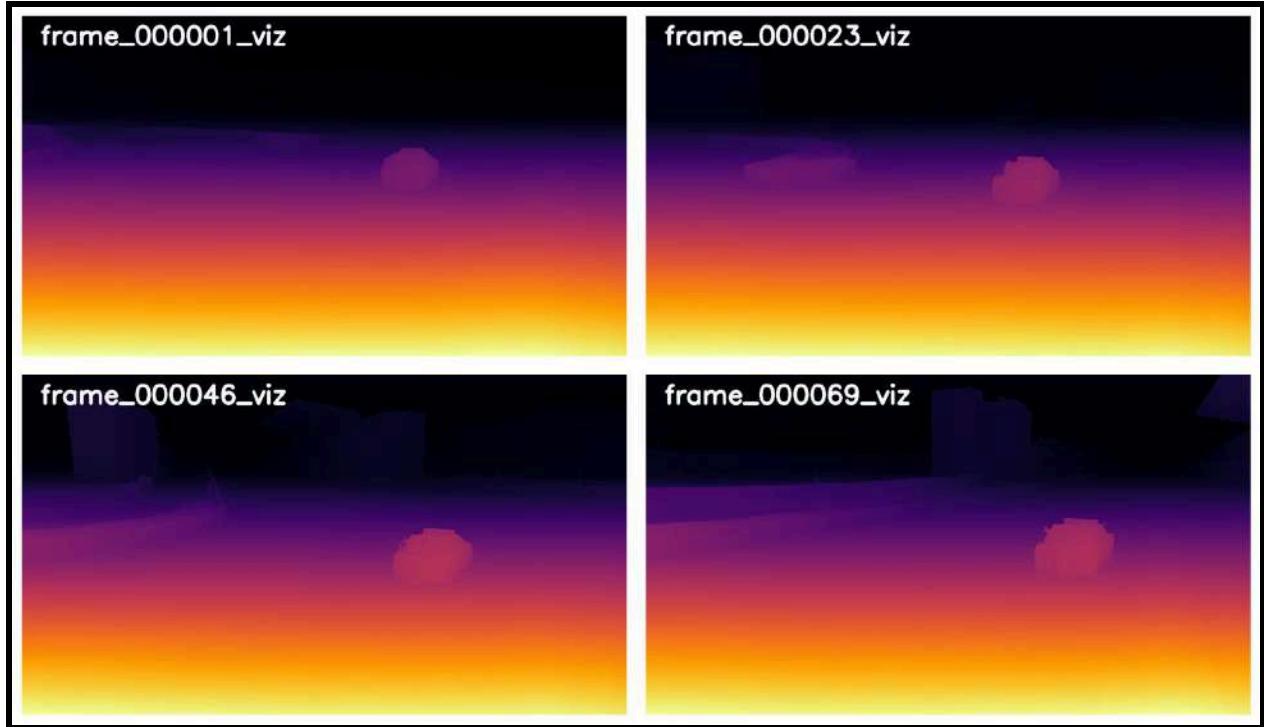


Fig 5: Composite image of 4 equally-spaced frames, showing depth-maps generated by monocular depth vision using 'Depth-Anything-V2-Small'

After completing the last two steps, we know the exact 2D coordinates of the car within each frame and the depth of the car from the drone's camera lens in each frame. Using the 2D pixel values, the depth values and the drone's actual position from COLMAP data generated in previous steps, it calculates where the car exists in the 3D world. The depth estimation can be jittery, hence a temporal filter is applied to average the 3D positions over time to ensure that the car moves in a smooth trajectory. However, the scale of the 3D trajectory may not match the real world. The 3D trajectory created at this stage is on a 'relative' or 'arbitrary' scale. This is because COLMAP is mathematically "scale-invariant" and the monocular depth model predicts relative depth.

The next step in the pipeline involves representing static urban scenes and dynamic vehicles separately. The paper uses the dense detailed point cloud from earlier steps to initialize the Gaussians. It reconstructs a "clean" environmental backdrop using Static Scene Gaussian Splatting. To prevent moving vehicles from creating blurry "ghosting" artifacts, we implement a masking pipeline where 2D binary masks—derived from the vehicle tracks—are applied to every frame. These masks instruct the Splatfacto trainer to ignore dynamic pixels, forcing the 3DGS model to learn only the permanent features of the urban landscape, such as buildings, road textures, and vegetation. The result is a high-fidelity, static 3D representation of the city that serves as the "empty stage". **Fig 6a - 6c** given below show the details of the static 3D scenes created by 3DGS model from three different views.



Fig 6a: View-1 of the static 3D scenes created by 3DGS model



Fig 6b: View-2 of the static 3D scenes created by 3DGS model

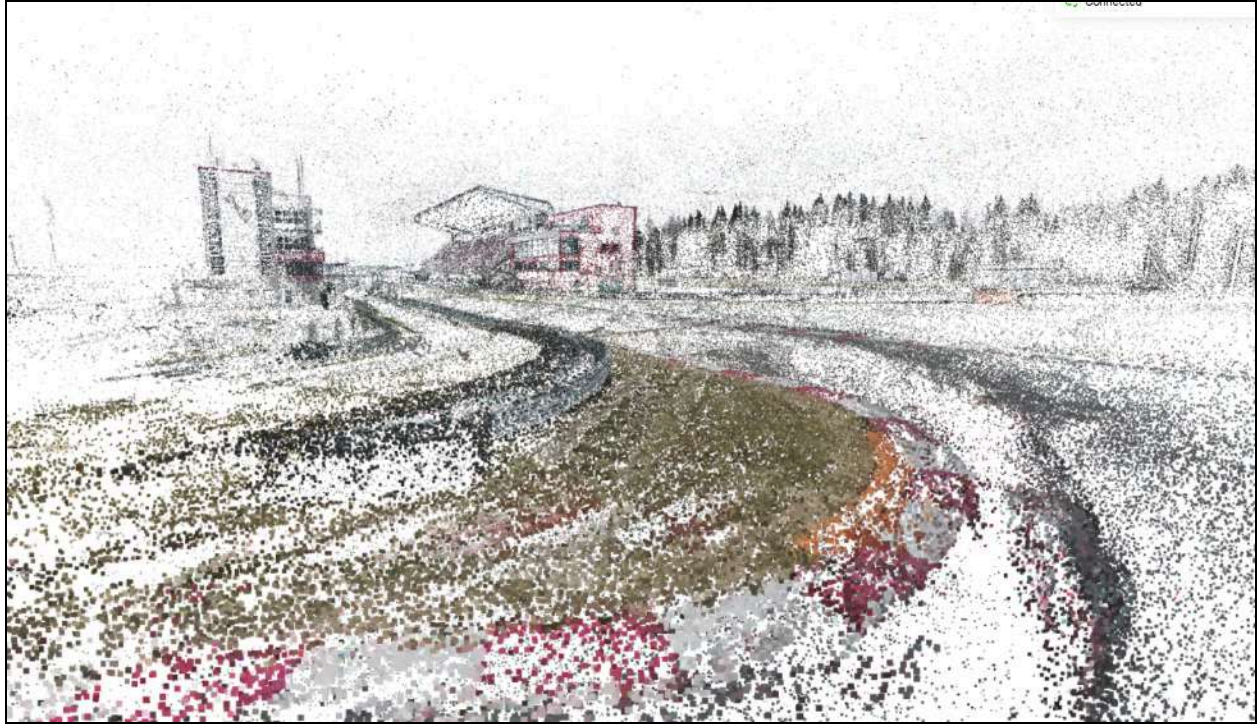


Fig 6c: View-3 of the static 3D scenes created by 3DGS model

These images show finer scenic details of the static scene like the shape of the track's kerbside and its specific color. These details were not visible in the COLMAP point-cloud seen in **Fig 3**.

The next stage focuses on Dynamic Vehicle Gaussian Splatting to reconstruct the moving vehicles themselves as modular assets. It isolates each vehicle by extracting cropped image patches based on their unique tracking IDs and training them in a "canonical" local coordinate system. This trick allows the 3DGS model to capture the fine details of each car as if it were a stationary object, avoiding the blurring caused by motion. These individual vehicle models were visualized via animation using Viser, an open-source, Python-based library for interactive 3D visualization. **Fig 7** given below shows the snapshot of the animated trajectory of the dynamic components of the scenes i.e. single vehicle as seen in Viser. The motion of the vehicle and its rough trajectory is evident from observing images 1 to 4 in **Fig 7**. The vehicle roughly follows a curved trajectory as seen in the input video.

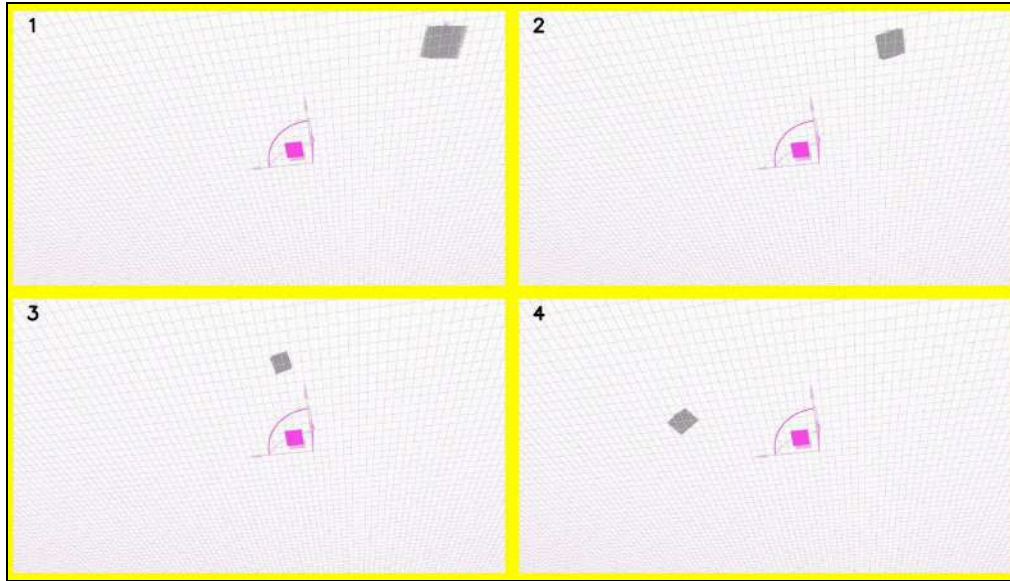


Fig 7: Animated trajectory of dynamic components (vehicle) as seen in Viser. The gray box represents the vehicle and its trajectory across 4 time-steps

These individual vehicle models are then coupled with the optimized 3D trajectories calculated in earlier stages. By applying the global position and orientation (heading) data to these canonical models, it can "re-insert" the vehicles into the static scene, enabling them to move realistically through the 3D environment in a final composite reconstruction. **Fig 8** given below shows the snapshot of the merged static and dynamic components across 4 time-steps.



Fig 8: Animated trajectory of dynamic components (vehicle) as seen in Viser. The gray box represents the vehicle and its trajectory is evident in these sequential images

As evident in **Fig 8**, there is mismatch in the coordinate systems of the static and dynamic components of the video. This is because the scale of the 3D trajectory does not match the scale of the real world . The 3D trajectory of the vehicle created is on a ‘relative’ or ‘arbitrary’ scale. This is because COLMAP is mathematically “scale-invariant” and the monocular depth model also predicts relative depth and not in any metric terms.

3. Methodology

3.1 Uplifting Trajectory to 3D World Coordinates

The 3D trajectory is extracted from the monocular drone video by combining per-frame YOLO detections with the camera intrinsics and extrinsics recovered from COLMAP. For each frame in which the target vehicle is detected, the bounding box bottom-centre pixel, corresponding to the tyre contact point, is lifted to 3D world coordinates using an analytical per-frame homography derived from the camera intrinsic matrix K and the camera-to-world transform $c2w$. This homography maps any image pixel directly to its corresponding point on the estimated world ground plane $Y = \text{ground}_y$, where ground_y is found once by a ray convergence search across all training cameras. As an alternative, Depth Anything V2 is used to estimate metric depth to the car surface at the detection pixel, and the resulting 3D car-body point is then projected further along the same camera ray until it intersects the ground plane, correcting for the horizontal displacement caused by the oblique drone viewpoint. The raw per-frame 3D positions are post-processed with a three-stage filter, kinematic spike removal, median filtering, and moving average smoothing, to suppress detection outliers and produce a clean continuous path. All coordinates are expressed in COLMAP world space, and the dataparser transform from nerfstudio is applied to convert them into the normalised scene space used by the 3DGS viewer.

3.2 Sparse Scene Reconstruction using PyCOLMAP

This stage of the pipeline involves recovering the camera motion and sparse geometric structure of the environment. This is achieved through Structure-from-Motion (SfM) using the PyCOLMAP wrapper for the COLMAP framework. Since the drone footage is uncalibrated, the process begins with an incremental SfM approach to simultaneously estimate camera intrinsic parameters and extrinsic poses for every frame.

The OPENCV camera model is employed during reconstruction, as it effectively accounts for the radial and tangential lens distortions typical of wide-angle UAV cameras. This model estimates the focal length (f), principal point (c_x, c_y), and distortion coefficients (k_1, k_2, p_1, p_2), ensuring that the projected 2D vehicle detections can be accurately mapped back to the 3D scene geometry. The workflow consists of three primary phases:

- 1. Feature Extraction and Matching:** SIFT features are extracted from each frame to ensure robustness against the perspective shifts and rotational changes inherent in UAV

flight. Sequential matching strategy is used to establish point correspondences across the image sequence and filtering outliers via RANSAC.

2. **Incremental Reconstruction:** It begins with an initial image pair having sufficient baseline, and the scene is built incrementally by adding new images, triangulating 3D points, and performing image registrations.
3. **Bundle Adjustment:** To minimize the global re-projection error, periodic bundle adjustment is done. This optimizes both the sparse 3D point cloud and the camera parameters, providing the metric foundation required for the subsequent dense reconstruction.

3.3 Dense Scene Reconstruction using 3D Gaussian Splatting

The sparse geometry recovered by PyCOLMAP is used to perform the dense scene reconstruction using 3D Gaussian Splatting (3DGS). 3DGS represents the scene as a collection of millions of 3D Gaussian primitives. Each Gaussian is defined by a mean position (μ), a covariance matrix (Σ) representing its shape and orientation, an opacity (α), and view-dependent color coefficients (Spherical Harmonics).

The reconstruction process is executed within the nerfstudio framework, utilizing the following pipeline:

1. **Initialization:** The sparse point cloud serves as the initial seed for the Gaussian primitives. This ensures that the dense model is anchored to the same coordinate system as our camera poses.
2. **Optimization:** The primitives are optimized through a differentiable rasterization process. It minimizes the L_1 and D-SSIM loss between the rendered views and the original drone frames. During training, the Gaussians undergo adaptive density control - splitting in areas of high geometric detail and cloning in under-reconstructed regions.
3. **Refinement:** To handle the high-frequency textures of the urban environment (e.g. asphalt, road markings), the optimization continues until the PSNR (Peak Signal-to-Noise Ratio) stabilizes.

The resulting digital twin provides a photorealistic, continuous representation of the scene. This enables the integration of the vehicle trajectory by providing a visual and geometric reference for the road surface, against which the uplifted 3D coordinates can be validated.

3.4 Integrating Trajectory to Scene

The 3D trajectory points are converted from COLMAP coordinates into the coordinate system used by the nerfstudio 3DGS scene using a scale and rotation transform stored in

dataparser_transforms.json. Once converted, the trajectory is drawn directly inside the nerfstudio viewer on top of the rendered scene — as dots, a smooth line, and direction arrows — in the same browser window. This works by injecting the trajectory geometry into the same viewer session that displays the 3DGS scene, so no retraining is needed when the trajectory changes. The main problem is that the conversion between the two coordinate systems is not perfectly accurate — a small error in where the ground plane sits in COLMAP space gets amplified by the transform, and the trajectory ends up floating above the road rather than sitting on it.

4. Results of Extensions

4.1 Uplifting Trajectory to 3D World Coordinates

The pipeline used was different from [1]. This section is about the results from the estimation of camera intrinsic and extrinsics obtained using COLMAP. Since we are dealing with uncalibrated cameras, to get an idea of actual scene geometry, we need to have an estimation of these parameters. The result of this is the intrinsics of the camera model used, along with frame-wise extrinsics which represent the camera's translation and rotation with respect to global coordinates. These results can be interpreted in 2D by looking at how camera position changes, as shown in **Fig 9**.

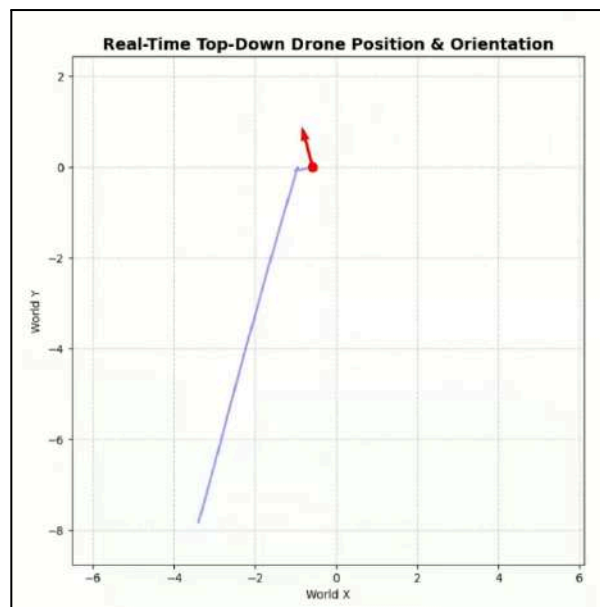


Fig 9: The Camera Position and Orientation retrieved (shown from a bird's eye view)

Testing for the 3D trajectory generated was done by comparing it to the one generated using Depth Anything V2 [4] on the same video. Many approaches use the depth inference model to judge how far the pixel is from the camera. In the first video, the car was moving in a straight line, and the results are shown in **Fig 10** (For reference: The actual vehicle path was a straight line).

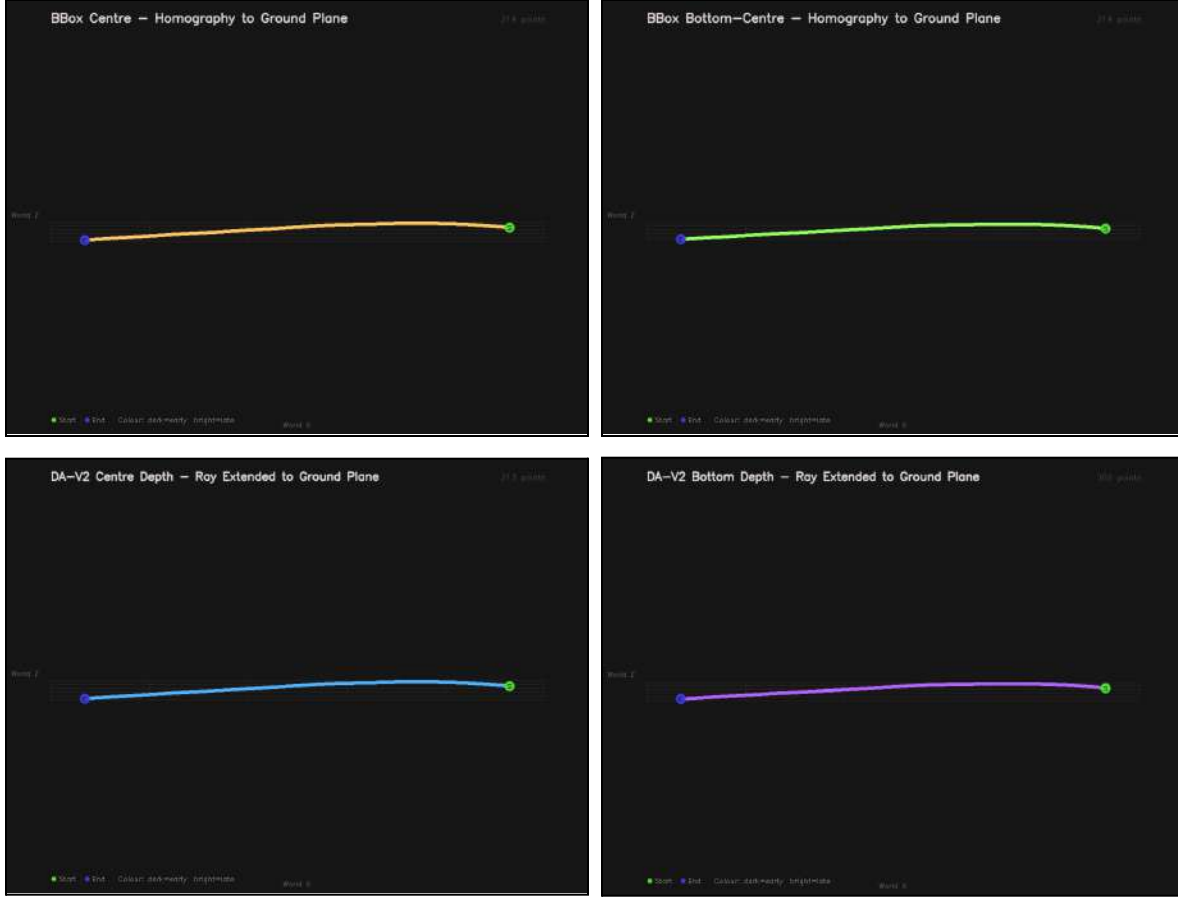


Fig 10: Different trajectory outputs based on the method used for Pixel $\rightarrow$ World conversion

As we can observe, **Fig 10** demonstrates that the method used by raycasting followed by projection on the ground plane works almost identical to the one using depth estimation with deep learning models. Further testing was done in other video sources and they all showed almost identical trajectories retrieved.

4.2 Sparse Scene Reconstruction using PyCOLMAP

The frames sampled from the video serve as inputs for structure-from-motion (SfM) which estimates the camera poses and generates a sparse point cloud that captures the essential geometry of the scene. It employs multi-view stereo (MVS) to create a detailed point cloud that reflects the intricate structure of the urban landscape. This dense point cloud forms the foundational model for the static components of the scene and is subsequently used to initialize the Gaussian field representation. **Fig 11** below shows the screen-shot of the detailed point cloud as viewed in Open3D viewer from four different views for one of the input videos used in our experiments.

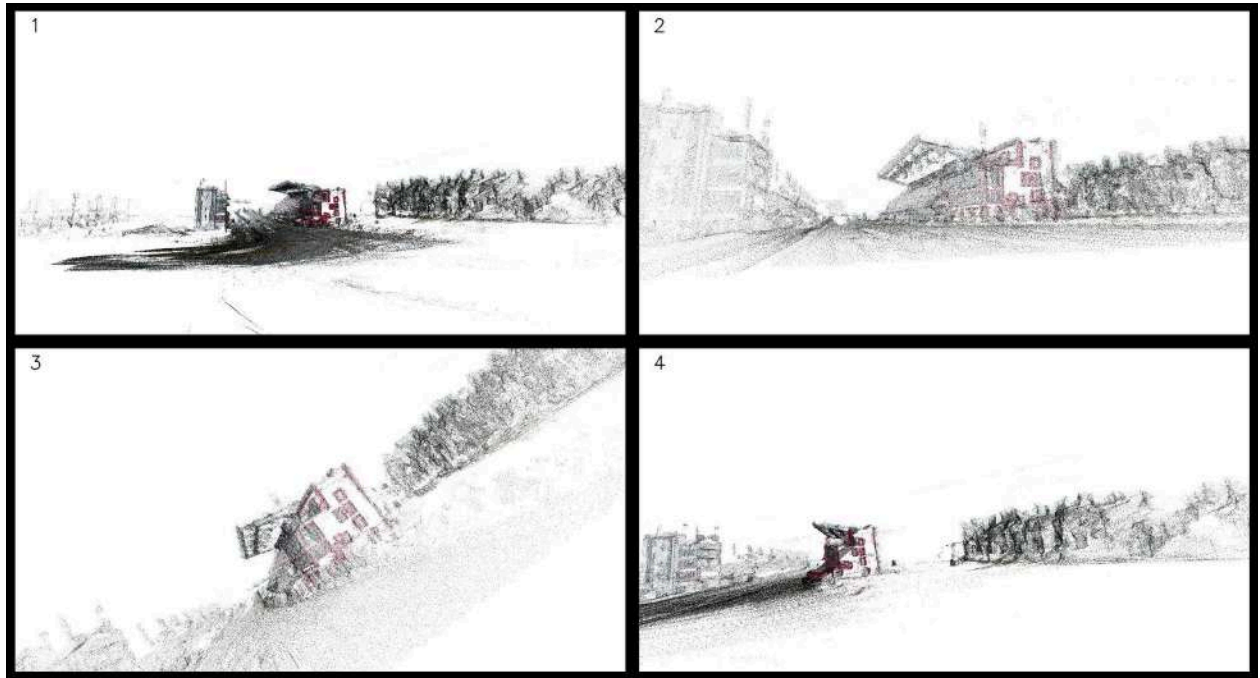


Fig 11: Screen-shot of detailed point cloud viewed in Open3D viewer from 4 different views

4.3 Dense Scene Reconstruction using 3D Gaussian Splatting

We also tried making some changes in the way the 3D Gaussian Splatted model is rendered. The motion of the vehicle results in artifacts in the reconstructed scene. But since most scenes are static, we can try to mitigate these artifacts. This was done by masking the car pixels using segmentation models. Two approaches were tried, one with inpainting and one without it. The results are shown in **Fig 12**.

But as we can see, the results are not good. The results are not as good as when we don't use any masking for vehicles, which is shown in **Fig 13**. Worth noting that all 3 methods used same time steps during training to maintain evaluation fairness.

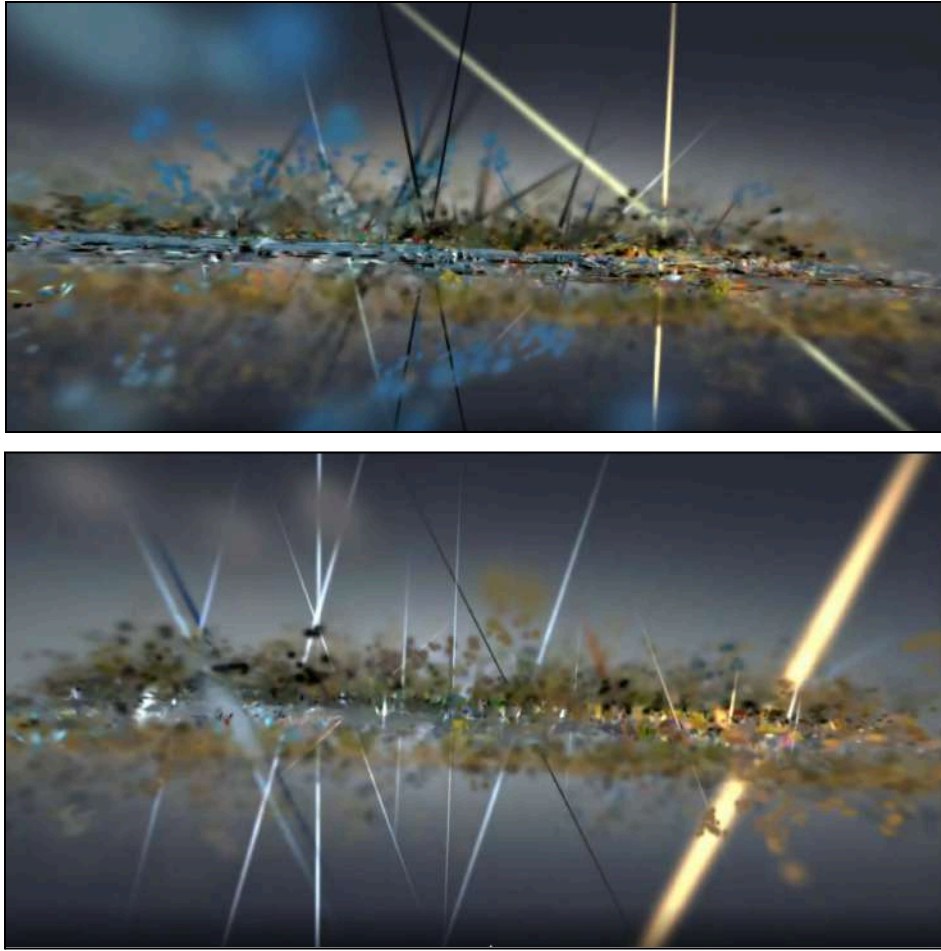


Fig 12: 3DGS without using In-painting (top) and with using In-painting (down)

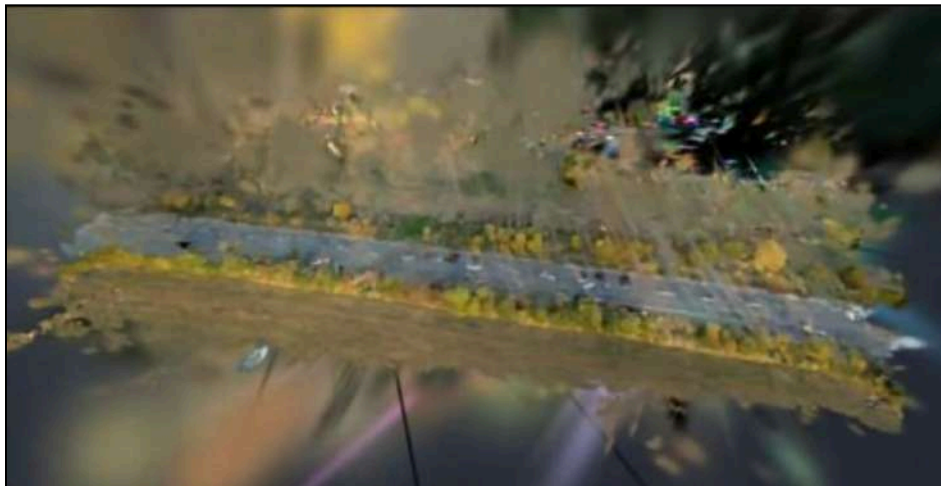


Fig 13: 3DGS Scene Reconstruction without any masking

4.4 Integrating Trajectory to Scene

Approaches for scene rendering with vehicle trajectories, such as [1], typically implement the entire process end-to-end, where the trajectory and scene representation are jointly optimised and therefore share a consistent coordinate system by construction. The approach explored here instead decouples scene reconstruction and trajectory lifting into two independent modules, with the motivation that each component can be developed, optimised, or replaced without retraining the other. The trajectory is extracted geometrically from camera intrinsics and extrinsics, and the static 3DGS scene is trained separately via splatfacto. The two are then combined in a shared viewer by applying the nerfstudio dataparser coordinate transform to map the trajectory into scene space.

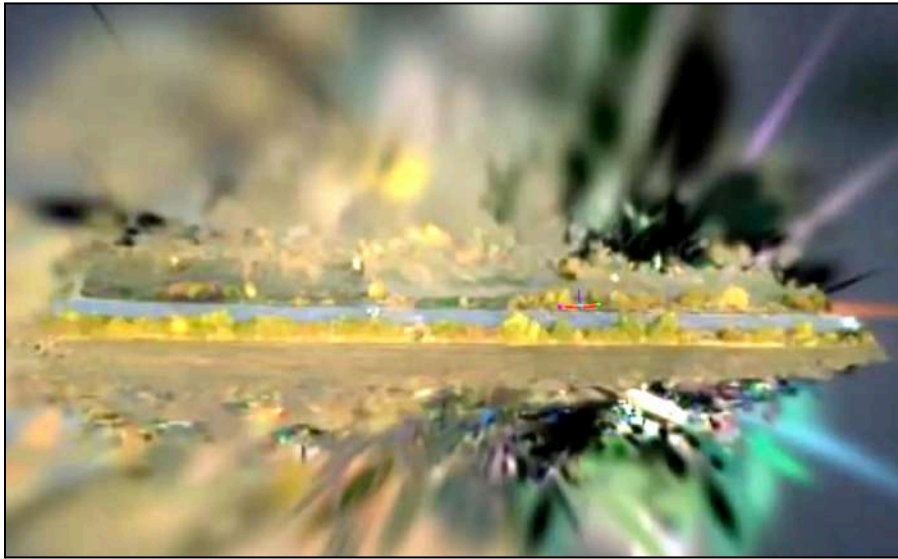


Fig 14: Tried injecting 3D Trajectory in the Reconstructed Scene

The results of this overlay are shown in **Fig 14**, where the trajectory can be seen floating above the road surface with incorrect vertical placement. The root cause is that the coordinate transform between COLMAP world space and nerfstudio's normalised scene space introduces a residual vertical offset that cannot be resolved analytically without knowing the exact elevation of the road surface in scene space — information that end-to-end methods like [1] never need to recover explicitly because it emerges from joint optimisation. Multiple alignment strategies were explored, including ground plane fitting from camera ray convergence, bounding box bottom-edge homography projection, and metric depth reverse-projection via Depth Anything V2 [4], none of which produced satisfactory alignment in the final scene.

5. Discussion

The individual components of the pipeline performed as expected. Camera localisation via COLMAP produced accurate per-frame intrinsics and extrinsics, YOLOv8 with BoT-SORT reliably tracked the target vehicle across frames with minimal identity switches, and the 3DGS reconstruction converged to a photorealistic digital twin of the scene. The top-down trajectory

visualisations confirmed that the homography-based ground projection produces geometrically consistent 2D paths that align with the drone camera coverage. However, the final integration step, overlaying the extracted trajectory within the 3DGS viewer, did not produce satisfactory alignment. The core difficulty is a coordinate system mismatch between the COLMAP reconstruction space and the normalised scene space used internally by nerfstudio, compounded by uncertainty in the true ground plane elevation. While the dataparser transform provided by nerfstudio nominally maps between these spaces, small errors in the estimated ground Y level are amplified after transformation, causing the trajectory to float visibly above the road surface. Resolving this would require either a direct depth query against the trained Gaussian primitives at the projected pixel location, or a ground control point correspondence between known world coordinates and their scene-space equivalents, neither of which was implemented within the scope of this project.

References:

- [1] Huang, Debao, Hanyang Liu, Ningli Xu, and Rongjun Qin. "Dynamic Urban Scene Modeling with 3D Gaussian Splatting from UAV Full Motion Videos." *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences* 48 (2025): 649-656.
- [2] Schonberger, Johannes L., and Jan-Michael Frahm. "Structure-from-motion revisited." In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 4104-4113. 2016.
- [3] Mildenhall, Ben, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. "Nerf: Representing scenes as neural radiance fields for view synthesis." *Communications of the ACM* 65, no. 1 (2021): 99-106.
- [4] Yang, Lihe, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. "Depth anything v2." *Advances in Neural Information Processing Systems* 37 (2024): 21875-21911.
- [5] Kerbl, Bernhard, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. "3d gaussian splatting for real-time radiance field rendering." *ACM Trans. Graph.* 42, no. 4 (2023): 139-1.
- [6] Tancik, Matthew, Ethan Weber, Evonne Ng, Ruilong Li, Brent Yi, Terrance Wang, Alexander Kristoffersen et al. "Nerfstudio: A modular framework for neural radiance field development." In *ACM SIGGRAPH 2023 conference proceedings*, pp. 1-12. 2023.
- [7] Nir Aharon, Roy Orfaig, Ben-Zion Bobrovsky, "BoT-SORT: Robust Associations Multi-pedestrian Tracking." *arXIV preprint arXIV:2206.14651*. 2023.